



**"RANKED 45
IN INDIA**

#University Category



**WORLD
UNIVERSITY
RANKINGS**

NO.1 PVT. UNIVERSITY IN
ACADEMIC REPUTATION IN INDIA



ACCREDITED GRADE 'A'
BY NAAC



PERFECT SCORE OF 150/150 AS A TESTAMENT
TO EXCEPTIONAL E-LEARNING METHODS

1



Applied Machine Learning

Unit 06 – Lecture 14: Decision Trees and Attribute Selection

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

Entropy, information gain, ID3, and reading a tree as rules

Topic focus: **a model made of questions, chosen by arithmetic**

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 4–5, 7, 14.; Zhang et al., *Dive into Deep Learning*, Ch. 5.

Trees
oooo

Entropy
ooooo

IG
ooooooooo

The ID3 algorithm
ooooooooo

Rules
ooo

Gini
ooooo

Ratio
ooooooooo

Scale
oo

Pruning
ooooo

Summary
oo

Quick Links

[What a tree is](#)

[Entropy](#)

[Information gain](#)

[ID3](#)

[Tree to rules](#)

[Gini](#)

[Gain ratio](#)

[Overfitting](#)

[Summary](#)

Agenda

What a decision tree is

Entropy

Information gain

The ID3 algorithm

Converting a tree to rules

The Gini index

Gain ratio, and the bias of information gain

Trees need no feature scaling

Overfitting, and how to stop it

Summary

How this lecture works

Every idea appears twice

1. **By hand** — fourteen days of weather, four columns of words. Small enough that every logarithm can be checked with a pen.
2. **In code** — the same arithmetic, then the same arithmetic again inside `scikit-learn`, so you can see the library is doing exactly this and nothing magic.

Commit before you look

Five slides ask you to predict a number before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

The centrepiece is **the full ID3 computation on fourteen rows**: $H(S)$, the gain of all four attributes, the recursion, the tree. It is the most examinable arithmetic in Unit VI.

One table, three datasets, one seed

The data

The fourteen-row play-tennis table, typed out rather than drawn; scikit-learn's bundled `load_iris`, `load_wine` and `load_breast_cancer`; and one noisy synthetic set fixed by `make_classification(..., random_state=0)`. Nothing is downloaded.

The randomness

One seed: `np.random.default_rng(0)` and `random_state=0` in every split, shuffle and tree. Where the *spread* is the measurement — 2000 entropy-vs-Gini contests, 200 bootstrap root splits, 30 repeated 70/30 splits — the seeds run 0, 1, 2, ... from that same one.

Why this matters to you

Every number in this deck is printed by one script. Run the code as written and you get the same numbers — which is the only reason to believe a number on a slide.

Where we are

The previous lecture stored the data

k -nearest neighbours keeps every training row and answers a query by asking who is nearby. All the work happens at prediction time, and the explanation is “these five rows looked like you”.

This lecture stores questions instead

A decision tree keeps no rows at all. It keeps a sequence of tests. Answer them and you arrive at a prediction, and **the path you took is the explanation**.

Today's one question

Given a set of labelled rows, *which question should I ask first?* Everything else in the lecture is machinery for answering that.

And the answer is a number

Measure how mixed up a set of labels is. Try every attribute. Keep the one that reduces the mixed-up-ness the most. That measure is **entropy**, the reduction is **information gain**, and the algorithm that repeats it is **ID3**.

A model made of questions

Internal node = a test. Branch = an answer. Leaf = a class label.

To classify a new row

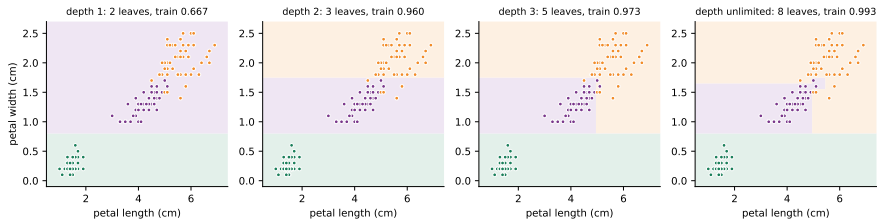
Start at the root. Answer the question.
Follow the matching branch. Repeat until
you hit a leaf. Read the label off the leaf.

What is *not* in that description

No distance. No coefficients. No
requirement that anything be numeric —
the example this lecture is built on has
four attributes and **every one of them is
a word.**

A decision tree is a nested set of IF–THEN rules a human can read out loud. Every other
property of trees — no scaling needed, categorical data handled natively, spectacular overfitting
— **follows from that one sentence.**

Drawn: a tree is a recursive partition



Two features of `load_iris`. Each question “is $x_j \leq t$?” is a cut perpendicular to an axis, always inside a region an earlier cut produced. Depth 1 separates one species perfectly; depth 2 reaches 0.960 training accuracy with three leaves; unlimited depth reaches 0.993 with eight. **The thin slivers on the right are what overfitting looks like geometrically** — and cross-validated, depth 2 and unlimited depth both score 0.9267.

Induction is greedy, and it has to be

The search space is hopeless

There are astronomically many trees consistent with a given table, and finding the smallest one is **NP-complete**.

So every practical algorithm is greedy

Pick the single best question *now*, split the data, recurse on each part, **never reconsider**. The family is called *top-down induction of decision trees*.

The only thing left to choose

What “best question” means. That choice is an **attribute selection measure**, and Han, Kamber and Pei give three:

- **information gain** — ID3
- **gain ratio** — C4.5
- **Gini index** — CART

All three are in this lecture

Information gain comes first, because the other two are best understood as answers to problems it has.

The table everything is built on

Quinlan's fourteen-day weather record. Build it yourself; do not download it.

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

rows 14 Yes 9 No 5

Outlook levels: Overcast, Rain, Sunny

Temperature levels: Cool, Hot, Mild

Humidity levels: High, Normal

Wind levels: Strong, Weak

The table everything is built on

Quinlan's fourteen-day weather record. Build it yourself; do not download it.

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

rows 14 Yes 9 No 5

Outlook levels: Overcast, Rain, Sunny

Temperature levels: Cool, Hot, Mild

Humidity levels: High, Normal

Wind levels: Strong, Weak

Four attributes, one class column — and a Day identifier that is **not** an attribute. Remember it is there. A whole section of this lecture is about what happens when you forget.

The question entropy answers

A node of the tree holds some rows

If every row has the same label the node is **pure** and there is nothing left to learn. If the labels are evenly split the node is as **impure** as it can be: knowing a row landed here tells you nothing.

So we want a number that is

0 in the first case, maximal in the second, and smooth in between.

Shannon's answer

$$H = - \sum_{i=1}^m p_i \log_2 p_i, \quad 0 \log_2 0 = 0.$$

Two names, one formula

Han, Kamber and Pei write this $\text{Info}(D)$ and call it *the expected information needed to classify a tuple in D*. Both names appear in examination papers. **They are the same thing.**

By hand: entropy at the two extremes

A pure node

All rows “Yes”, so $p_{\text{Yes}} = 1$, $p_{\text{No}} = 0$:

$$H = -1 \log_2 1 - 0 \log_2 0 = \mathbf{0}.$$

No questions needed. You already know.

An evenly split binary node

$$H = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = \frac{1}{2} + \frac{1}{2} = \mathbf{1 \text{ bit}}.$$

Exactly one yes/no question, which is

obvious.

m equally likely classes

$$H = - \sum_{i=1}^m \frac{1}{m} \log_2 \frac{1}{m} = \log_2 m.$$

Three classes give 1.5850, four give 2, ten give 3.3219. **This is the ceiling.**

H is symmetric

9 Yes and 5 No has exactly the same entropy as 5 Yes and 9 No. Entropy measures *how mixed*, not *which class dominates*. Two nodes with equal entropy can predict opposite labels.

The same thing in code

```
def H(p):  
    return 0.0 if p in (0.0, 1.0) else -(p*log2(p) + (1-p)*log2(1-p))
```

entropy of a two-class node, $H(p) = -p \log_2 p - (1-p) \log_2 (1-p)$

p = 0.0000	H = 0.0000	pure
p = 0.0500	H = 0.2864	
p = 0.1000	H = 0.4690	
p = 0.2500	H = 0.8113	
p = 0.3333	H = 0.9183	
p = 0.5000	H = 1.0000	MAXIMUM
p = 0.6667	H = 0.9183	
p = 0.7500	H = 0.8113	
p = 0.9000	H = 0.4690	
p = 1.0000	H = 0.0000	pure

The same thing in code

```
def H(p):  
    return 0.0 if p in (0.0, 1.0) else -(p*log2(p) + (1-p)*log2(1-p))
```

entropy of a two-class node, $H(p) = -p \log_2 p - (1-p) \log_2 (1-p)$

p = 0.0000	H = 0.0000	pure
p = 0.0500	H = 0.2864	
p = 0.1000	H = 0.4690	
p = 0.2500	H = 0.8113	
p = 0.3333	H = 0.9183	
p = 0.5000	H = 1.0000	MAXIMUM
p = 0.6667	H = 0.9183	
p = 0.7500	H = 0.8113	
p = 0.9000	H = 0.4690	
p = 1.0000	H = 0.0000	pure

uniform over k classes: $H = \log_2(k)$

k = 2	H = 1.0000	$\log_2(k) = 1.0000$
k = 3	H = 1.5850	$\log_2(k) = 1.5850$
k = 4	H = 2.0000	$\log_2(k) = 2.0000$
k = 8	H = 3.0000	$\log_2(k) = 3.0000$
k = 10	H = 3.3219	$\log_2(k) = 3.3219$

Why the logarithm, and why base two

Think of it as a coding problem. A label of probability p deserves $\log_2(1/p)$ bits, and H is the average of those costs. So H is in **bits**: the average number of yes/no questions to pin down one label.

why base 2: H is the average number of yes/no questions per message

2 equally likely messages $\rightarrow \log_2(2) = 1.0$ bits, and 1 yes/no questions do identify one

4 equally likely messages $\rightarrow \log_2(4) = 2.0$ bits, and 2 yes/no questions do identify one

8 equally likely messages $\rightarrow \log_2(8) = 3.0$ bits, and 3 yes/no questions do identify one

16 equally likely messages $\rightarrow \log_2(16) = 4.0$ bits, and 4 yes/no questions do identify one

a biased two-class source, 1000 draws at $p(\text{Yes}) = 0.9$

observed $p(\text{Yes})$ 0.9040 H 0.4562 bits per draw

a naive 1-bit-per-draw code costs 1000 bits; the entropy bound is 456

Why the logarithm, and why base two

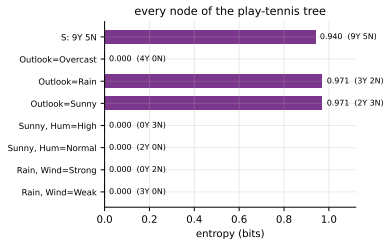
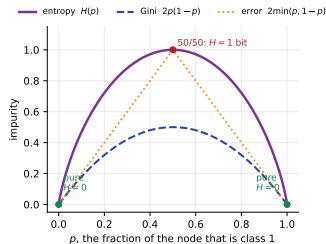
Think of it as a coding problem. A label of probability p deserves $\log_2(1/p)$ bits, and H is the average of those costs. So H is in **bits**: the average number of yes/no questions to pin down one label.

```
why base 2: H is the average number of yes/no questions per message
2 equally likely messages ->  $\log_2(2) = 1.0$  bits, and 1 yes/no questions do identify one
4 equally likely messages ->  $\log_2(4) = 2.0$  bits, and 2 yes/no questions do identify one
8 equally likely messages ->  $\log_2(8) = 3.0$  bits, and 3 yes/no questions do identify one
16 equally likely messages ->  $\log_2(16) = 4.0$  bits, and 4 yes/no questions do identify one
a biased two-class source, 1000 draws at  $p(\text{Yes}) = 0.9$ 
observed  $p(\text{Yes})$  0.9040   H 0.4562 bits per draw
a naive 1-bit-per-draw code costs 1000 bits; the entropy bound is 456
```

The last two lines are the whole definition

A thousand labels that are 90% “Yes” fit in 456 bits rather than 1000, because they are predictable. **Entropy measures unpredictability, and a node of a tree is exactly a place where we want the labels to be predictable.** Base e gives nats, base ten gives hartleys — every comparison in this lecture comes out the same.

Drawn: the shape of the three impurity measures



Left: binary entropy, with Gini and the misclassification error rescaled to the same peak. All three are zero at the ends and maximal at $p = \frac{1}{2}$; **the error rate is flat where the other two still discriminate**, which is why it makes a poor splitting criterion. Right: every node of the tree we are about to build, in creation order — root at 0.9403, two children at 0.9710, and all five leaves exactly 0.

Two equations, and that is the whole idea

The impurity that survives a split

$$\text{Info}_A(S) = \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} H(S_v) \quad (\text{weighted average of the children})$$

What the split removed

$$\text{Gain}(S, A) = H(S) - \text{Info}_A(S)$$

Two facts that make it sensible

Gain ≥ 0 always — splitting cannot raise the weighted average entropy. And $\text{Gain} = H(S)$

exa

Tofik Ali

Applied Machine Learning

Say it in words

Gain is **the number of bits of uncertainty** about the class that asking about A

School of Computer Science, UPES Dehradun

By hand: $H(S)$ for the fourteen days

Nine “Yes”, five “No”, so $p_{\text{Yes}} = 9/14 = 0.642857$ and $p_{\text{No}} = 5/14 = 0.357143$. The logarithms are $\log_2 0.642857 = -0.637430$ and $\log_2 0.357143 = -1.485427$.

$$H(S) = 0.642857 \cdot 0.637430 + 0.357143 \cdot 1.485427 = 0.409776 + 0.530510 = \mathbf{0.940286} \text{ bits}$$

S has 9 Yes and 5 No out of 14

$$\begin{aligned} H(S) &= -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) \\ &= -(0.642857)(-0.637430) - (0.357143)(-1.485427) \\ &= 0.409776 + 0.530510 = 0.940286 \text{ bits} \end{aligned}$$

$$\text{Gini}(S) = 1 - (9/14)^2 - (5/14)^2 = 0.459184$$

By hand: $H(S)$ for the fourteen days

Nine “Yes”, five “No”, so $p_{\text{Yes}} = 9/14 = 0.642857$ and $p_{\text{No}} = 5/14 = 0.357143$. The logarithms are $\log_2 0.642857 = -0.637430$ and $\log_2 0.357143 = -1.485427$.

$$H(S) = 0.642857 \cdot 0.637430 + 0.357143 \cdot 1.485427 = 0.409776 + 0.530510 = \mathbf{0.940286} \text{ bits}$$

S has 9 Yes and 5 No out of 14

$$\begin{aligned} H(S) &= -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) \\ &= -(0.642857)(-0.637430) - (0.357143)(-1.485427) \\ &= 0.409776 + 0.530510 = 0.940286 \text{ bits} \end{aligned}$$

$$\text{Gini}(S) = 1 - (9/14)^2 - (5/14)^2 = 0.459184$$

Predict first #1 — write your answer down now

Outlook splits the fourteen rows 4/5/5, Temperature 4/4/6, Humidity 7/7, Wind 6/8. **Rank all four by information gain, and name the winner**, before we compute a single one of them. Keep the paper: we come back to it in four slides.

By hand: $\text{Gain}(S, \text{Outlook})$, slowly

value	rows	Yes	No	entropy of that child
Overcast	4	4	0	$H = -1 \log_2 1 = 0$ (pure)
Rain	5	3	2	$H = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} = 0.970951$
Sunny	5	2	3	$H = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.970951$

The last two are equal: 2 Yes out of 5 and 3 Yes out of 5 are equally mixed. Now the weighted average, then the subtraction:

$$\text{Info}_{\text{Outlook}}(S) = \frac{4}{14}(0) + \frac{5}{14}(0.970951) + \frac{5}{14}(0.970951) = \frac{10}{14} \times 0.970951 = 0.693536$$

$$\text{Gain}(S, \text{Outlook}) = 0.940286 - 0.693536 = \mathbf{0.246750}$$

The same thing in code

```
def info_after(rows, attr):  
    n = len(rows)  
    return sum(len(sub)/n * entropy(sub) for _, sub in partition(rows, attr))  
print(entropy(TENNIS) - info_after(TENNIS, "Outlook"))
```

Information gain of Outlook, step by step

Outlook = Overcast 4 rows 4 Yes 0 No H = 0.000000

Outlook = Rain 5 rows 3 Yes 2 No H = 0.970951

Outlook = Sunny 5 rows 2 Yes 3 No H = 0.970951

Info_Outlook(S) = (4/14)(0.0000) + (5/14)(0.9710) + (5/14)(0.9710)
 = 0.693536

Gain(S, Outlook) = 0.940286 - 0.693536 = 0.246750

The same thing in code

```
def info_after(rows, attr):  
    n = len(rows)  
    return sum(len(sub)/n * entropy(sub) for _, sub in partition(rows, attr))  
print(entropy(TENNIS) - info_after(TENNIS, "Outlook"))
```

Information gain of Outlook, step by step

Outlook = Overcast 4 rows 4 Yes 0 No H = 0.000000

Outlook = Rain 5 rows 3 Yes 2 No H = 0.970951

Outlook = Sunny 5 rows 2 Yes 3 No H = 0.970951

Info_Outlook(S) = $(4/14)(0.0000) + (5/14)(0.9710) + (5/14)(0.9710)$
 = 0.693536

Gain(S, Outlook) = $0.940286 - 0.693536 = 0.246750$

Line for line, the same six numbers we put on the board. **Three lines of Python and a dictionary of subsets is the entire attribute selection measure** — there is nothing else in it.

By hand: the other three at speed

Same three steps each time: entropy of every child, weighted average, subtract from $H(S)$.

attribute	entropy of each child			$\text{Info}_A(S)$
Temperature 4/4/6	Cool 0.811278	Hot 1.000000	Mild 0.918296	0.911063
Humidity 7/7	High 0.985228	Normal 0.591673		0.788450
Wind 6/8	Strong 1.000000	Weak 0.811278		0.892159

The middle column is the arithmetic, so do one of them in full:

$$\text{Info}_{\text{Temperature}} = \frac{4}{14}(0.811278) + \frac{4}{14}(1.000000) + \frac{6}{14}(0.918296) = 0.911063$$

By hand: the other three at speed

Same three steps each time: entropy of every child, weighted average, subtract from $H(S)$.

attribute	entropy of each child			$\text{Info}_A(S)$
Temperature 4/4/6	Cool 0.811278	Hot 1.000000	Mild 0.918296	0.911063
Humidity 7/7	High 0.985228	Normal 0.591673		0.788450
Wind 6/8	Strong 1.000000	Weak 0.811278		0.892159

The middle column is the arithmetic, so do one of them in full:

$$\text{Info}_{\text{Temperature}} = \frac{4}{14}(0.811278) + \frac{4}{14}(1.000000) + \frac{6}{14}(0.918296) = 0.911063$$

Subtract each from $H(S) = 0.940286$: $\text{Gain}(\text{Temperature}) = \mathbf{0.029223}$,
 $\text{Gain}(\text{Humidity}) = \mathbf{0.151836}$, $\text{Gain}(\text{Wind}) = \mathbf{0.048127}$.

All three, machine-checked

Information gain of Temperature

Temperature = Cool 4 rows 3 Yes 1 No H = 0.811278

Temperature = Hot 4 rows 2 Yes 2 No H = 1.000000

Temperature = Mild 6 rows 4 Yes 2 No H = 0.918296

Info_Temperature(S) = 0.911063 Gain = 0.940286 - 0.911063 = 0.029223

Information gain of Humidity

Humidity = High 7 rows 3 Yes 4 No H = 0.985228

Humidity = Normal 7 rows 6 Yes 1 No H = 0.591673

Info_Humidity(S) = 0.788450 Gain = 0.940286 - 0.788450 = 0.151836

Information gain of Wind

Wind = Strong 6 rows 3 Yes 3 No H = 1.000000

Wind = Weak 8 rows 6 Yes 2 No H = 0.811278

Info_Wind(S) = 0.892159 Gain = 0.940286 - 0.892159 = 0.048127

All three, machine-checked

Information gain of Temperature

Temperature = Cool 4 rows 3 Yes 1 No H = 0.811278

Temperature = Hot 4 rows 2 Yes 2 No H = 1.000000

Temperature = Mild 6 rows 4 Yes 2 No H = 0.918296

Info_Temperature(S) = 0.911063 Gain = 0.940286 - 0.911063 = 0.029223

Information gain of Humidity

Humidity = High 7 rows 3 Yes 4 No H = 0.985228

Humidity = Normal 7 rows 6 Yes 1 No H = 0.591673

Info_Humidity(S) = 0.788450 Gain = 0.940286 - 0.788450 = 0.151836

Information gain of Wind

Wind = Strong 6 rows 3 Yes 3 No H = 1.000000

Wind = Weak 8 rows 6 Yes 2 No H = 0.811278

Info_Wind(S) = 0.892159 Gain = 0.940286 - 0.892159 = 0.048127

Twelve child entropies, three weighted averages, three subtractions. Every number matches the board. **Now look at your ranking.**

Predict first #1 — the reveal

Four attributes, four gains. You wrote down a ranking four slides ago.

Which one does ID3 put at the root, and by how much?

Predict first #1 — the reveal

Four attributes, four gains. You wrote down a ranking four slides ago.

Which one does ID3 put at the root, and by how much?

attribute	Info_A(S)	Gain	rank
Outlook	0.693536	0.246750	1
Temperature	0.911063	0.029223	4
Humidity	0.788450	0.151836	2
Wind	0.892159	0.048127	3

ID3 splits the root on Outlook
runner-up Humidity is behind by 0.094914

Predict first #1 — the reveal

Four attributes, four gains. You wrote down a ranking four slides ago.

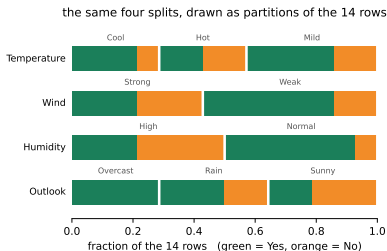
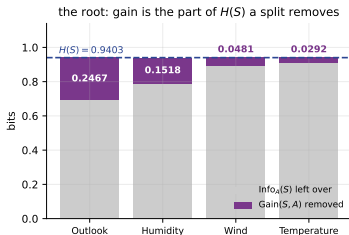
Which one does ID3 put at the root, and by how much?

attribute	Info_A(S)	Gain	rank
Outlook	0.693536	0.246750	1
Temperature	0.911063	0.029223	4
Humidity	0.788450	0.151836	2
Wind	0.892159	0.048127	3

ID3 splits the root on Outlook
runner-up Humidity is behind by 0.094914

Most people get the winner. Very few get Temperature in last place — it *feels* like the most relevant thing about a day of tennis, and it is the **worst** of the four questions to ask first.

Drawn: where the gain comes from



Left: each bar is $H(S) = 0.9403$ split into the part the question removes (purple, the gain) and the part it leaves behind (grey, $\text{Info}_A(S)$). Right: the same four splits as partitions of the fourteen rows, green Yes and orange No. **The Overcast block is entirely green** — that is where Outlook's gain comes from. Every Temperature block is a mixture, which is why its gain is 0.0292.

Why Outlook wins — and the commonest mistake

The reason is one number

Outlook is the **only attribute with a pure child**. Four Overcast days, four “Yes”, entropy 0.

Read the definition again

An attribute earns gain by producing children that are less mixed than the parent, and a **pure child is the most any child can contribute**. That is the sentence to reproduce in an examination.

Do not compare $\text{Info}_A(S)$ and pick the largest

You want the *smallest* Info, equivalently the *largest* gain. This sign error is the single commonest mistake in the examination, and it hands you Temperature — the worst of the four.

ID3 — Iterative Dichotomiser 3, Quinlan 1986

ID3(S , attributes)

1. If every row of S has the same class c , return a leaf labelled c .
2. If attributes is empty, return a leaf labelled with the **majority** class of S .
3. Otherwise let $A^* = \arg \max_A \text{Gain}(S, A)$.
4. Create a node testing A^* . For each value v of A^* : let S_v be the rows with $A^* = v$; if S_v is empty attach a leaf with the majority class of S ; otherwise attach **ID3**(S_v , attributes $\setminus \{A^*\}$).
5. Return the node.

Used once per path

After splitting on A , every row below has the same value of A . A second split would be useless.

Splits are multiway

k values give k branches, not two. CART does not do this, and it matters.

No pruning at all

It grows until every leaf is pure or the attributes run out. That is a problem, and it has its own section.

Step 3, by hand: the Sunny branch

Five Sunny days: D1, D2, D8 (No) and D9, D11 (Yes). Two Yes, three No, so $H = 0.970951$. Outlook is used up; three attributes remain.

```
Outlook = Sunny      : 5 rows, 2 Yes 3 No, H = 0.970951 -> recurse
  Gain(S_sunny, Temperature) = 0.970951 - 0.400000 = 0.570951
  Gain(S_sunny, Humidity    ) = 0.970951 - 0.000000 = 0.970951
  Gain(S_sunny, Wind        ) = 0.970951 - 0.950978 = 0.019973
  winner: Humidity, and it is a PERFECT split
    Humidity = High      3 rows  0 Yes 3 No   H = 0.0000
    Humidity = Normal    2 rows  2 Yes 0 No   H = 0.0000
```

Step 3, by hand: the Sunny branch

Five Sunny days: D1, D2, D8 (No) and D9, D11 (Yes). Two Yes, three No, so $H = 0.970951$. Outlook is used up; three attributes remain.

```

Outlook = Sunny      : 5 rows, 2 Yes 3 No, H = 0.970951 -> recurse
Gain(S_sunny, Temperature) = 0.970951 - 0.400000 = 0.570951
Gain(S_sunny, Humidity   ) = 0.970951 - 0.000000 = 0.970951
Gain(S_sunny, Wind       ) = 0.970951 - 0.950978 = 0.019973
winner: Humidity, and it is a PERFECT split
    Humidity = High      3 rows  0 Yes 3 No   H = 0.0000
    Humidity = Normal    2 rows  2 Yes 0 No   H = 0.0000
  
```

0.970951 is the *entire* entropy of the node

Gain = $H(S_{\text{sunny}})$ means Info = 0 means **both children are pure**, so the branch is finished in one step. Three high-humidity sunny days: no tennis. Two normal-humidity sunny days: tennis.

Predict first #2 — the Rain branch

Five Rain days: D4, D5, D10 (Yes) and D6, D14 (No). Three Yes, two No, so again $H = 0.970951$. Three attributes left.

Which one does ID3 choose here — and is it perfect too?

Predict first #2 — the Rain branch

Five Rain days: D4, D5, D10 (Yes) and D6, D14 (No). Three Yes, two No, so again $H = 0.970951$. Three attributes left.

Which one does ID3 choose here — and is it perfect too?

```
Outlook = Rain      : 5 rows, 3 Yes 2 No, H = 0.970951  -> recurse
Gain(S_rain, Temperature) = 0.970951 - 0.950978 = 0.019973
Gain(S_rain, Humidity   ) = 0.970951 - 0.950978 = 0.019973
Gain(S_rain, Wind       ) = 0.970951 - 0.000000 = 0.970951
winner: Wind, and it is a PERFECT split
  Wind = Strong  2 rows  0 Yes 2 No   H = 0.0000
  Wind = Weak   3 rows  3 Yes 0 No   H = 0.0000
```

Predict first #2 — the Rain branch

Five Rain days: D4, D5, D10 (Yes) and D6, D14 (No). Three Yes, two No, so again $H = 0.970951$. Three attributes left.

Which one does ID3 choose here — and is it perfect too?

```
Outlook = Rain      : 5 rows, 3 Yes 2 No, H = 0.970951  -> recurse
Gain(S_rain, Temperature) = 0.970951 - 0.950978 = 0.019973
Gain(S_rain, Humidity   ) = 0.970951 - 0.950978 = 0.019973
Gain(S_rain, Wind       ) = 0.970951 - 0.000000 = 0.970951
winner: Wind, and it is a PERFECT split
  Wind = Strong  2 rows  0 Yes 2 No   H = 0.0000
  Wind = Weak   3 rows  3 Yes 0 No   H = 0.0000
```

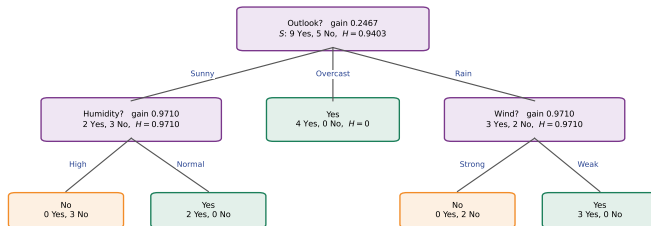
Rain and a strong wind: no tennis. Rain and a weak wind: tennis. And note Temperature and Humidity **tie** at 0.019973 — ties happen, and an implementation must break them somehow, usually by attribute order.

The whole algorithm, in nine lines

```
def id3(rows, attrs):
    yes, no = counts(rows)
    if yes == 0 or no == 0: # pure -> leaf
        return Leaf("Yes" if yes else "No")
    if not attrs: # out of attributes -> majority
        return Leaf("Yes" if yes >= no else "No")
    best = max(attrs, key=lambda a: gain(rows, a))
    return Node(best, {v: id3(sub, [a for a in attrs if a != best])
                       for v, sub in partition(rows, best)})
```

```
[9 Yes, 5 No H=0.9403] split on Outlook (gain 0.2467)
Outlook=Overcast: [4 Yes, 0 No H=0.0000] --> Yes
Outlook=Rain: [3 Yes, 2 No H=0.9710] split on Wind (gain 0.9710)
    Wind=Strong: [0 Yes, 2 No H=0.0000] --> No
    Wind=Weak: [3 Yes, 0 No H=0.0000] --> Yes
Outlook=Sunny: [2 Yes, 3 No H=0.9710] split on Humidity (gain 0.9710)
    Humidity=High: [0 Yes, 3 No H=0.0000] --> No
    Humidity=Normal: [2 Yes, 0 No H=0.0000] --> Yes
leaves: 5
```

Drawn: the tree ID3 builds



5 leaves, so 5 IF-THEN rules; every leaf is pure, so the tree reproduces all 14 training rows

Class counts, entropy and the winning gain at every node. Depth 2, five leaves, every leaf pure. Temperature **never appears** — not because temperature is irrelevant to tennis, but because *given* the outlook, humidity and wind already account for these fourteen days. A greedy algorithm answers “what is the best question to ask next”, not “which variables matter”.

And if the attribute is a number?

ID3 as stated is categorical only. C4.5 and CART turn a numeric attribute into a binary one: sort the distinct values, take the midpoint of each adjacent pair as a candidate threshold, and score $x \leq t$ against $x > t$ with the *same* formula. With d distinct values, $d - 1$ candidates.

```
vals = np.unique(petal_length)
cands = (vals[:-1] + vals[1:]) / 2 # the midpoints
best = max(cands, key=lambda t: gain_of_threshold(t))
```

```
a continuous attribute: where does the threshold come from?
43 distinct petal-length values -> 42 candidate midpoints
H(all 150 rows, setosa vs rest) = 0.918296
best midpoint 2.45 cm, gain 0.918296 (a perfect split)
scikit-learn's threshold: 2.45 -- the same number
```

And if the attribute is a number?

ID3 as stated is categorical only. C4.5 and CART turn a numeric attribute into a binary one: sort the distinct values, take the midpoint of each adjacent pair as a candidate threshold, and score $x \leq t$ against $x > t$ with the *same* formula. With d distinct values, $d - 1$ candidates.

```
vals = np.unique(petal_length)
cands = (vals[:-1] + vals[1:]) / 2 # the midpoints
best = max(cands, key=lambda t: gain_of_threshold(t))
```

```
a continuous attribute: where does the threshold come from?
43 distinct petal-length values -> 42 candidate midpoints
H(all 150 rows, setosa vs rest) = 0.918296
best midpoint 2.45 cm, gain 0.918296 (a perfect split)
scikit-learn's threshold: 2.45 -- the same number
```

Forty-two candidates, one of them perfect, and the library lands on the same 2.45 cm. One asymmetry to remember: **a numeric attribute can be split again further down at a different threshold**; a categorical one cannot.

scikit-learn implements CART, not ID3

Same fourteen rows, one-hot encoded because it will not take strings, with `criterion="entropy"`.

```
t = DecisionTreeClassifier(criterion="entropy", random_state=0).fit(X, y)
print/export_text(t, feature_names=names)
```

```
|--- Outlook=Overcast <= 0.50
| |--- Humidity=Normal <= 0.50
| | |--- Outlook=Rain <= 0.50
| | | |--- class: 0
| | |--- Outlook=Rain > 0.50
| | | |--- Wind=Strong <= 0.50
| | | | |--- class: 1
| | | |--- Wind=Strong > 0.50
| | | | |--- class: 0
| | |--- Humidity=Normal > 0.50
| | |--- Wind=Weak <= 0.50
| | | |--- Temperature=Cool <= 0.50
| | | | |--- class: 1
| | | |--- Temperature=Cool > 0.50
| | | | |--- class: 0
| | |--- Wind=Weak > 0.50
| | | |--- class: 1
|--- Outlook=Overcast > 0.50
| |--- class: 1
depth 4, leaves 7, training accuracy 1.0000
our ID3 tree: depth 2, leaves 5, training accuracy 1.0000
```

Why it is a different tree — and it is only one thing

root feature: Outlook=Overcast (root impurity 0.940286, matches $H(S) = 0.940286$)

the binary questions available at the root, scored by information gain:

Outlook=Overcast gain 0.226000

Humidity=Normal gain 0.151836

Humidity=High gain 0.151836

Outlook=Sunny gain 0.102244

Wind=Weak gain 0.048127

the best BINARY question is Outlook=Overcast; the best 3-WAY question is Outlook
that is the whole difference: ID3 splits multiway, CART splits in two.

Why it is a different tree — and it is only one thing

root feature: Outlook=Overcast (root impurity 0.940286, matches $H(S) = 0.940286$)

the binary questions available at the root, scored by information gain:

Outlook=Overcast gain 0.226000

Humidity=Normal gain 0.151836

Humidity=High gain 0.151836

Outlook=Sunny gain 0.102244

Wind=Weak gain 0.048127

the best BINARY question is Outlook=Overcast; the best 3-WAY question is Outlook
that is the whole difference: ID3 splits multiway, CART splits in two.

The measure is identical

Root impurity 0.940286 — **our** $H(S)$, exactly.

Only the *shape* of the allowed split differs.

Restricted to two branches the best is 0.226000
against the three-way 0.246750, and the tree
needs an extra level to recover the rest.

Say what you actually ran

scikit-learn has **no ID3 and no C4.5**. It
has an optimised CART, it always splits in
two, and it will not take categorical columns.
Do not write “I ran ID3 in scikit-learn” in a
lab report.

By hand: five leaves, five rules

A tree with L leaves is L IF–THEN rules. Walk root to leaf, conjoin the tests you passed through, and the leaf label is the consequent.

R_1 : IF Outlook = Overcast	THEN Yes	(4 rows)
R_2 : IF Outlook = Rain AND Wind = Strong	THEN No	(2 rows)
R_3 : IF Outlook = Rain AND Wind = Weak	THEN Yes	(3 rows)
R_4 : IF Outlook = Sunny AND Humidity = High	THEN No	(3 rows)
R_5 : IF Outlook = Sunny AND Humidity = Normal	THEN Yes	(2 rows)

Five rules, nine conditions, $4 + 2 + 3 + 3 + 2 = 14$ rows. They are **mutually exclusive** (a row satisfies exactly one) and **exhaustive** (every row satisfies one), because the leaves partition the data.

The same thing in code — and why bother

one IF-THEN rule per root-to-leaf path

R1: IF Outlook = Overcast THEN PlayTennis = Yes (4 rows)

R2: IF Outlook = Rain AND Wind = Strong THEN PlayTennis = No (2 rows)

R3: IF Outlook = Rain AND Wind = Weak THEN PlayTennis = Yes (3 rows)

R4: IF Outlook = Sunny AND Humidity = High THEN PlayTennis = No (3 rows)

R5: IF Outlook = Sunny AND Humidity = Normal THEN PlayTennis = Yes (2 rows)

rules: 5 conditions in total: 9

the rule set reproduces 14 of 14 training rows

The same thing in code — and why bother

```
one IF-THEN rule per root-to-leaf path
R1: IF Outlook = Overcast THEN PlayTennis = Yes      (4 rows)
R2: IF Outlook = Rain AND Wind = Strong THEN PlayTennis = No      (2 rows)
R3: IF Outlook = Rain AND Wind = Weak THEN PlayTennis = Yes      (3 rows)
R4: IF Outlook = Sunny AND Humidity = High THEN PlayTennis = No      (3 rows)
R5: IF Outlook = Sunny AND Humidity = Normal THEN PlayTennis = Yes      (2 rows)
rules: 5      conditions in total: 9
the rule set reproduces 14 of 14 training rows
```

They read better

“If it is overcast, play” is a sentence a non-specialist can check against their own experience. A picture of a tree is not.

They prune independently

Drop a condition from *one* rule without disturbing the others. You cannot do that to a tree: removing a test removes the whole subtree below it.

They reorder

Sort by accuracy or coverage, put a default rule at the end, and the set often shortens again.

Rule simplification, measured on iris

A depth-3 tree on load_iris, read out as rules.

leaves 5, so 5 rules; the tree scores 0.9733 on all 150 rows

R1: IF petal width (cm) <= 0.80 THEN setosa (50 rows, impurity 0.000)

R2: IF petal width (cm) > 0.80 AND petal width (cm) <= 1.75 AND petal length (cm) <= 4.95 THEN
versicolor (48 rows, impurity 0.041)

R3: IF petal width (cm) > 0.80 AND petal width (cm) <= 1.75 AND petal length (cm) > 4.95 THEN
virginica (6 rows, impurity 0.444)

R4: IF petal width (cm) > 0.80 AND petal width (cm) > 1.75 AND petal length (cm) <= 4.85 THEN
virginica (3 rows, impurity 0.444)

R5: IF petal width (cm) > 0.80 AND petal width (cm) > 1.75 AND petal length (cm) > 4.85 THEN
virginica (43 rows, impurity 0.000)

R4 and R5 predict the SAME class, so the last condition can be dropped:

IF petal width > 0.80 AND petal width > 1.75 THEN virginica (46 rows)

rules before simplification 5, after 4

the 4 simplified rules still classify 146 of 150 iris rows correctly

Rule simplification, measured on iris

A depth-3 tree on load_iris, read out as rules.

```
leaves 5, so 5 rules; the tree scores 0.9733 on all 150 rows
R1: IF petal width (cm) <= 0.80 THEN setosa (50 rows, impurity 0.000)
R2: IF petal width (cm) > 0.80 AND petal width (cm) <= 1.75 AND petal length (cm) <= 4.95 THEN
    versicolor (48 rows, impurity 0.041)
R3: IF petal width (cm) > 0.80 AND petal width (cm) <= 1.75 AND petal length (cm) > 4.95 THEN
    virginica (6 rows, impurity 0.444)
R4: IF petal width (cm) > 0.80 AND petal width (cm) > 1.75 AND petal length (cm) <= 4.85 THEN
    virginica (3 rows, impurity 0.444)
R5: IF petal width (cm) > 0.80 AND petal width (cm) > 1.75 AND petal length (cm) > 4.85 THEN
    virginica (43 rows, impurity 0.000)
R4 and R5 predict the SAME class, so the last condition can be dropped:
    IF petal width > 0.80 AND petal width > 1.75 THEN virginica (46 rows)
rules before simplification 5, after 4
the 4 simplified rules still classify 146 of 150 iris rows correctly
```

The split that produced R_4 and R_5 separated three rows from forty-three and then **predicted the same class on both sides**. It bought the splitter some impurity and the prediction nothing, so it can go. **A leaf is a rule and a rule is a leaf — but only the rule form can be edited one condition at a time.** That asymmetry is why C4.5 converts to rules *before* pruning.

By hand: the Gini index

$\text{Gini}(S) = 1 - \sum_{i=1}^m p_i^2$ — the probability that two rows drawn at random have *different* labels.

$$\text{Gini}(S) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 1 - 0.413265 - 0.127551 = \mathbf{0.459184}$$

For Outlook: $\text{Gini}(\text{Overcast}) = 0$ and

$\text{Gini}(\text{Rain}) = \text{Gini}(\text{Sunny}) = 1 - 0.36 - 0.16 = 0.48$, so

$$\text{Gini}_{\text{Outlook}}(S) = \frac{4}{14}(0) + \frac{10}{14}(0.48) = 0.342857$$

$$\text{Gini gain} = 0.459184 - 0.342857 = \mathbf{0.116327}$$

Zero for a pure node, maximal for an even split — but the maximum for two classes is 0.5, not

1. **And there is no logarithm in it**, which is why implementations like it.

The same four attributes, scored both ways

the same four attributes scored with the Gini index instead

Gini(S) = 0.459184

attribute	Gini_A(S)	Gini gain	rank	(entropy rank)
Outlook	0.342857	0.116327	1	1
Temperature	0.440476	0.018707	4	4
Humidity	0.367347	0.091837	2	2
Wind	0.428571	0.030612	3	3

entropy order: Outlook > Humidity > Wind > Temperature

Gini order: Outlook > Humidity > Wind > Temperature

same winner? True same full ordering? True

The same four attributes, scored both ways

the same four attributes scored with the Gini index instead

Gini(S) = 0.459184

attribute	Gini_A(S)	Gini gain	rank	(entropy rank)
Outlook	0.342857	0.116327	1	1
Temperature	0.440476	0.018707	4	4
Humidity	0.367347	0.091837	2	2
Wind	0.428571	0.030612	3	3

entropy order: Outlook > Humidity > Wind > Temperature

Gini order: Outlook > Humidity > Wind > Temperature

same winner? True same full ordering? True

All the numbers are smaller

Gini lives on a shorter scale. But the **ranking is identical in all four positions**. That is the usual outcome, and it is why the choice of criterion is rarely the interesting decision.

Never compare across measures

0.2467 and 0.1163 are **the same split measured in different units**. Only compare gains computed with the same measure on the same node.

CART splits in two, so it must choose a subset

A three-valued attribute cannot give three branches in a binary tree: the algorithm must send a *subset* of the values left. With k values there are $2^{k-1} - 1$ distinct cuts, since a subset and its complement agree.

```
CART splits BINARY, so for a 3-valued attribute it must choose a subset
{Overcast      } | rest    4 vs 10 rows    Gini gain 0.102041
{Rain          } | rest    5 vs  9 rows    Gini gain 0.002041
{Sunny         } | rest    5 vs  9 rows    Gini gain 0.065533
(a subset and its complement are the same cut, so 3 values give 3 cuts)
best binary cut on Outlook: {Overcast} | {Rain, Sunny}
the 3-way multiway split ID3 uses scores 0.116327
```

CART splits in two, so it must choose a subset

A three-valued attribute cannot give three branches in a binary tree: the algorithm must send a *subset* of the values left. With k values there are $2^{k-1} - 1$ distinct cuts, since a subset and its complement agree.

```
CART splits BINARY, so for a 3-valued attribute it must choose a subset
{Overcast      } | rest    4 vs 10 rows    Gini gain 0.102041
{Rain          } | rest    5 vs  9 rows    Gini gain 0.002041
{Sunny         } | rest    5 vs  9 rows    Gini gain 0.065533
(a subset and its complement are the same cut, so 3 values give 3 cuts)
best binary cut on Outlook: {Overcast} | {Rain, Sunny}
the 3-way multiway split ID3 uses scores 0.116327
```

The best binary cut scores 0.102041 against the multiway split's 0.116327. **Binary trees pay a little at each node and buy it back with depth** — which is exactly the depth-4-against-depth-2 result we saw two sections ago, now with a number attached to it.

How often do entropy and Gini actually disagree?

“They usually give the same tree” is repeated everywhere. Measure it.

```
2-attribute contests on random tables : 1990 of 1999 agree (99.55%)
iris      root split: entropy f3<=0.8000   gini f3<=0.8000   same? True
iris      5-fold accuracy entropy 0.9333   gini 0.9333   difference +0.0000
iris      on held-out rows the two trees agree on 97.78% of them
wine      root split: entropy f6<=1.5750   gini f12<=755.0000 same? False
wine      5-fold accuracy entropy 0.9495   gini 0.9273   difference +0.0222
wine      on held-out rows the two trees agree on 92.59% of them
breast_cancer root split: entropy f22<=105.9500 gini f20<=16.7950 same? False
breast_cancer 5-fold accuracy entropy 0.9384   gini 0.9262   difference +0.0122
breast_cancer on held-out rows the two trees agree on 92.98% of them
200 bootstrap root splits of breast_cancer identical: 150 (75.0%)
but the two stumps still label 98.40% of rows the same on average
```

How often do entropy and Gini actually disagree?

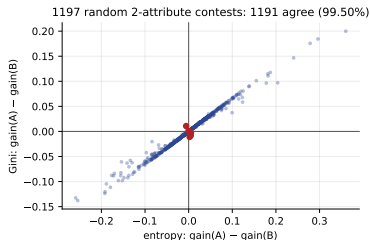
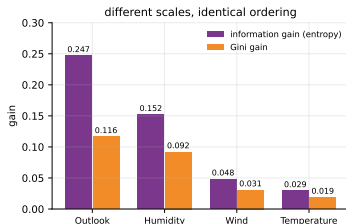
“They usually give the same tree” is repeated everywhere. Measure it.

```
2-attribute contests on random tables : 1990 of 1999 agree (99.55%)
iris      root split: entropy f3<=0.8000   gini f3<=0.8000   same? True
iris      5-fold accuracy entropy 0.9333   gini 0.9333   difference +0.0000
iris      on held-out rows the two trees agree on 97.78% of them
wine      root split: entropy f6<=1.5750   gini f12<=755.0000 same? False
wine      5-fold accuracy entropy 0.9495   gini 0.9273   difference +0.0222
wine      on held-out rows the two trees agree on 92.59% of them
breast_cancer root split: entropy f22<=105.9500 gini f20<=16.7950 same? False
breast_cancer 5-fold accuracy entropy 0.9384   gini 0.9262   difference +0.0122
breast_cancer on held-out rows the two trees agree on 92.98% of them
200 bootstrap root splits of breast_cancer identical: 150 (75.0%)
but the two stumps still label 98.40% of rows the same on average
```

Three statements, in order

When one attribute is clearly better, they agree — 99.55% of two thousand contests, and every disagreement was a near-tie. **When there are thousands of near-tied candidates, they often differ** — the breast_cancer root split changed on 25% of bootstrap resamples, far more than the folklore admits. **And it barely matters** — those stumps still labelled 98.4% of rows the same. Choose on cost, not on principle: Gini has no logarithm and is the default.

Drawn: entropy against Gini, two thousand times



Left: the four tennis attributes scored both ways — different heights, identical ordering. Right: two thousand random two-attribute contests, entropy's margin horizontally against Gini's vertically. Almost every point sits in the lower-left or upper-right quadrant, meaning the two agreed. **The red points** — 0.45% of them — **all sit close to the origin**, where the two attributes were nearly tied anyway.

Information gain has a flaw, and it is not subtle

It systematically prefers attributes with **many values**.

Why, in one sentence

An attribute that cuts the data into many small pieces produces children that are purer *almost by accident*. In the limit, one value per row gives fourteen children of one row each, every one of them pure.

And the tennis table has such a column

It is called Day. It has been sitting in the file since the first slide of the lecture, and nothing in the algorithm knows it is an identifier rather than an attribute.

Predict first #3: what information gain does Day score, and how does it compare with Outlook's 0.246750?

Predict first #3 — scoring the Day column

Fourteen values in fourteen rows, so every child holds one row and has entropy 0.

Therefore $\text{Info}_{\text{Day}}(S) = \sum_v \frac{1}{14} \times 0 = 0$, and the gain is the whole of $H(S)$.

Predict first #3 — scoring the Day column

Fourteen values in fourteen rows, so every child holds one row and has entropy 0.

Therefore $\text{Info}_{\text{Day}}(S) = \sum_v \frac{1}{14} \times 0 = 0$, and the gain is the whole of $H(S)$.

```
now score the Day column, which is a row identifier
```

```
Day has 14 distinct values in 14 rows
```

```
Day = D1    1 row    H = 0.0000
```

```
Day = D10   1 row    H = 0.0000
```

```
Day = D11   1 row    H = 0.0000
```

```
... all 14 subsets have one row each, so every H = 0
```

```
Info_Day(S) = 0.000000
```

```
Gain(S, Day) = 0.940286 - 0.000000 = 0.940286 <-- the largest possible
```

```
Gini gain(S, Day) = 0.459184 (Gini has exactly the same bias)
```

```
best real attribute: Gain(S, Outlook) = 0.246750
```

```
Day beats it by a factor of 3.81
```

Predict first #3 — scoring the Day column

Fourteen values in fourteen rows, so every child holds one row and has entropy 0.

Therefore $\text{Info}_{\text{Day}}(S) = \sum_v \frac{1}{14} \times 0 = 0$, and the gain is the whole of $H(S)$.

```
now score the Day column, which is a row identifier
```

```
Day has 14 distinct values in 14 rows
```

```
Day = D1    1 row    H = 0.0000
```

```
Day = D10   1 row    H = 0.0000
```

```
Day = D11   1 row    H = 0.0000
```

```
... all 14 subsets have one row each, so every H = 0
```

```
Info_Day(S) = 0.000000
```

```
Gain(S, Day) = 0.940286 - 0.000000 = 0.940286 <-- the largest possible
```

```
Gini gain(S, Day) = 0.459184 (Gini has exactly the same bias)
```

```
best real attribute: Gain(S, Outlook) = 0.246750
```

```
Day beats it by a factor of 3.81
```

The largest gain any attribute can possibly have, $3.81\times$ the best real one — and the Gini index has exactly the same problem, scoring the whole of $\text{Gini}(S) = 0.459184$. This is not a defect of entropy. It is a defect of *unnormalised impurity reduction*.

What ID3 does with it — exactly what it is told

what happens if you let ID3 see the Day column

[9 Yes, 5 No H=0.9403] split on Day (gain 0.9403)

Day=D1: [0 Yes, 1 No H=0.0000] --> No

Day=D10: [1 Yes, 0 No H=0.0000] --> Yes

Day=D11: [1 Yes, 0 No H=0.0000] --> Yes

... (14 branches, one per row)

leaves 14, depth 1, conditions per rule 1

training rows reproduced: 14 of 14 (perfect)

a new row D15 (Sunny, Cool, Normal, Weak) matches 0 of the 14 rules

the 4-attribute tree classifies it as: Yes

What ID3 does with it — exactly what it is told

```
what happens if you let ID3 see the Day column
[9 Yes, 5 No H=0.9403] split on Day (gain 0.9403)
  Day=D1: [0 Yes, 1 No H=0.0000] --> No
  Day=D10: [1 Yes, 0 No H=0.0000] --> Yes
  Day=D11: [1 Yes, 0 No H=0.0000] --> Yes
  ... (14 branches, one per row)
leaves 14, depth 1, conditions per rule 1
training rows reproduced: 14 of 14 (perfect)
a new row D15 (Sunny, Cool, Normal, Weak) matches 0 of the 14 rules
the 4-attribute tree classifies it as: Yes
```

Read the last two lines twice

A depth-1 tree with fourteen leaves, **perfect on the training set**, that **matches none of its own rules on a new day**. This is the cleanest demonstration of overfitting in the whole course: a model that is right about everything it has seen and has nothing whatever to say about anything else.

The fix: divide by what the split itself costs

C4.5: normalise by the entropy of the split itself, ignoring the labels

$$\text{SplitInfo}_A(S) = - \sum_v \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}, \quad \text{GainRatio}(S, A) = \frac{\text{Gain}(S, A)}{\text{SplitInfo}_A(S)}$$

A two-way even split has $\text{SplitInfo} = 1$ and is not penalised at all. A fourteen-way split has $\log_2 14 = 3.807$ and is divided by nearly four.

$$\text{SplitInfo}_{\text{Outlook}} = -\frac{5}{14} \log_2 \frac{5}{14} - \frac{4}{14} \log_2 \frac{4}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 1.577406$$

$$\text{SplitInfo}_{\text{Humidity}} = -\frac{7}{14} \log_2 \frac{7}{14} - \frac{7}{14} \log_2 \frac{7}{14} = 1.000000$$

$$\text{SplitInfo}_{\text{Day}} = \log_2 14 = 3.807355$$

Predict first #4: does dividing by that fix the Day problem? $0.940286/3.807355$ against $0.246750/1.577406$ — do the division before the next slide.

Predict first #4 — the reveal

```
gain ratio = gain / split information
```

attribute	Gain	SplitInfo	GainRatio	gain rank	ratio rank
Outlook	0.246750	1.577406	0.156428	1	1
Temperature	0.029223	1.556657	0.018773	4	4
Humidity	0.151836	1.000000	0.151836	2	2
Wind	0.048127	0.985228	0.048849	3	3

```
information gain picks Outlook; gain ratio picks Outlook
```

Predict first #4 — the reveal

```
gain ratio = gain / split information
```

attribute	Gain	SplitInfo	GainRatio	gain rank	ratio rank
Outlook	0.246750	1.577406	0.156428	1	1
Temperature	0.029223	1.556657	0.018773	4	4
Humidity	0.151836	1.000000	0.151836	2	2
Wind	0.048127	0.985228	0.048849	3	3

```
information gain picks Outlook; gain ratio picks Outlook
```

```
SplitInfo(Day) =  $\log_2(14) = 3.807355$ 
```

```
GainRatio(Day) =  $0.940286 / 3.807355 = 0.246966$ 
```

```
that is a 73.7% cut, from 3.81x the best real attribute to 1.58x
```

```
but at 14 rows Day still ranks FIRST on gain ratio: 0.246966 vs Outlook 0.156428
```

Predict first #4 — the reveal

gain ratio = gain / split information

attribute	Gain	SplitInfo	GainRatio	gain rank	ratio rank
Outlook	0.246750	1.577406	0.156428	1	1
Temperature	0.029223	1.556657	0.018773	4	4
Humidity	0.151836	1.000000	0.151836	2	2
Wind	0.048127	0.985228	0.048849	3	3

information gain picks Outlook; gain ratio picks Outlook

SplitInfo(Day) = $\log_2(14) = 3.807355$

GainRatio(Day) = $0.940286 / 3.807355 = 0.246966$

that is a 73.7% cut, from 3.81x the best real attribute to 1.58x

but at 14 rows Day still ranks FIRST on gain ratio: 0.246966 vs Outlook 0.156428

Textbooks say gain ratio fixes the bias. On this table it does not

It *reduces* it enormously — a 73.7% cut — but 0.246966 still beats 0.156428, so **the identifier would still be chosen**. Do not claim otherwise in an answer script. Also notice what the correction did to the honest attributes: Outlook 0.156428 against Humidity 0.151836, a gap of 0.005 where plain gain had 0.095. The three-way split was penalised; the two-way one was not.

Where the correction does bite: more rows

SplitInfo of an identifier is $\log_2 n$, which grows with the table. Its gain is stuck at $H(S)$, which does not. So replicate the table.

copies	rows	Gain(RowID)	SplitInfo	GainRatio(RowID)	best real GR	Day wins?
1	14	0.940286	3.807355	0.246966	0.156428	YES
2	28	0.940286	4.807355	0.195593	0.156428	YES
5	70	0.940286	6.129283	0.153409	0.156428	no
10	140	0.940286	7.129283	0.131891	0.156428	no
20	280	0.940286	8.129283	0.115667	0.156428	no

$\log_2(\text{rows})$ grows; the gain of a perfect identifier does not.

Where the correction does bite: more rows

SplitInfo of an identifier is $\log_2 n$, which grows with the table. Its gain is stuck at $H(S)$, which does not. So replicate the table.

copies	rows	Gain(RowID)	SplitInfo	GainRatio(RowID)	best real GR	Day wins?
1	14	0.940286	3.807355	0.246966	0.156428	YES
2	28	0.940286	4.807355	0.195593	0.156428	YES
5	70	0.940286	6.129283	0.153409	0.156428	no
10	140	0.940286	7.129283	0.131891	0.156428	no
20	280	0.940286	8.129283	0.115667	0.156428	no

$\log_2(\text{rows})$ grows; the gain of a perfect identifier does not.

the 14-row table	gain picks Day	(14 leaves)	gain ratio picks Day	(14 leaves)
5 copies, 70 rows	gain picks Day	(70 leaves)	gain ratio picks Outlook	(5 leaves)

Where the correction does bite: more rows

SplitInfo of an identifier is $\log_2 n$, which grows with the table. Its gain is stuck at $H(S)$, which does not. So replicate the table.

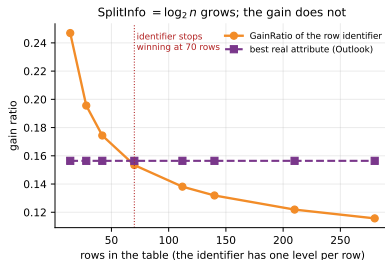
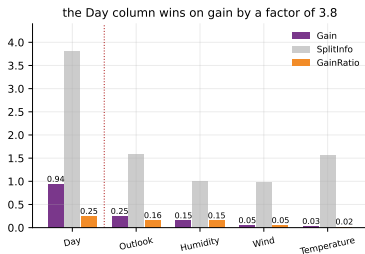
copies	rows	Gain(RowID)	SplitInfo	GainRatio(RowID)	best real GR	Day wins?
1	14	0.940286	3.807355	0.246966	0.156428	YES
2	28	0.940286	4.807355	0.195593	0.156428	YES
5	70	0.940286	6.129283	0.153409	0.156428	no
10	140	0.940286	7.129283	0.131891	0.156428	no
20	280	0.940286	8.129283	0.115667	0.156428	no

$\log_2(\text{rows})$ grows; the gain of a perfect identifier does not.

the 14-row table	gain picks Day	(14 leaves)	gain ratio picks Day	(14 leaves)
5 copies, 70 rows	gain picks Day	(70 leaves)	gain ratio picks Outlook	(5 leaves)

At seventy rows it is unambiguous: information gain still builds the useless seventy-leaf identifier tree, and **gain ratio recovers the correct five-leaf tree**. That is the result to remember — with the $n = 14$ caveat attached to it.

Drawn: the identifier, penalised



Left: gain, split information and gain ratio for the four real attributes and Day. Day's gain bar is the tallest possible; its split-information bar is 3.807; dividing brings it down to 0.247. Right: repeating the table so the identifier has more levels. Its gain ratio falls like $H(S)/\log_2 n$ while the best real attribute stays flat at 0.156. **They cross at seventy rows.**

Two honest footnotes before we leave this

the same identifier, one-hot encoded, given to scikit-learn

4 attributes (10 columns) train 1.0000 depth 4 leaves 7 7-fold CV 0.6429

+ Day column (24 columns) train 1.0000 depth 3 leaves 5 7-fold CV 0.6429

one-hot turns a 14-way question into 14 separate yes/no questions,
so CART cannot isolate one row in a single split and the damage is
milder than ID3's -- but the CV score is unchanged either way.

Two honest footnotes before we leave this

the same identifier, one-hot encoded, given to scikit-learn

```
4 attributes (10 columns)    train 1.0000 depth 4 leaves 7 7-fold CV 0.6429
```

```
+ Day column (24 columns)   train 1.0000 depth 3 leaves 5 7-fold CV 0.6429
```

one-hot turns a 14-way question into 14 separate yes/no questions,
so CART cannot isolate one row in a single split and the damage is
milder than ID3's -- but the CV score is unchanged either way.

C4.5 does not use the ratio blindly

It first discards any attribute whose *plain* gain is below average, then maximises the ratio among the rest — because dividing by a tiny SplitInfo can promote a useless near-constant attribute.

The catastrophe is a *multiway* one

Binary-split CART peels off one row at a time, so a one-hot identifier did not move the CV score here. The lesson survives either way, and it is short: **drop identifier columns before you fit anything.**

A split is a question about order, not magnitude

Every question is “is $x_j \leq t$?”. Apply any strictly increasing g : then

$x_j \leq t \iff g(x_j) \leq g(t)$, so **the same rows go left** with a different number printed on the node. Same partitions available, same tree.

```
a tree needs no feature scaling: any increasing transform is a no-op
tree raw                leaves 16 depth 6 test 0.906433 same predictions? True
tree StandardScaler     leaves 16 depth 6 test 0.906433 same predictions? True
tree log1p (nonlinear, increasing) leaves 16 depth 6 test 0.906433 same predictions? True
tree cube (nonlinear, increasing) leaves 16 depth 6 test 0.906433 same predictions? True
tree x -> -x (DECREASING) leaves 19 depth 6 test 0.900585 same predictions?
False
the decreasing case: the SAME partitions are available, but the
greedy search breaks its ties the other way round. Check the root:
root feature raw 22 thr +106.1000 | negated 22 thr -106.1000 | same 2 groups? True
```

A split is a question about order, not magnitude

Every question is “is $x_j \leq t$?”. Apply any strictly increasing g : then

$x_j \leq t \iff g(x_j) \leq g(t)$, so **the same rows go left** with a different number printed on the node. Same partitions available, same tree.

```
a tree needs no feature scaling: any increasing transform is a no-op
tree raw                      leaves 16 depth 6 test 0.906433 same predictions? True
tree StandardScaler           leaves 16 depth 6 test 0.906433 same predictions? True
tree log1p (nonlinear, increasing) leaves 16 depth 6 test 0.906433 same predictions? True
tree cube (nonlinear, increasing) leaves 16 depth 6 test 0.906433 same predictions? True
tree x -> -x (DECREASING)      leaves 19 depth 6 test 0.900585 same predictions?
False
the decreasing case: the SAME partitions are available, but the
greedy search breaks its ties the other way round. Check the root:
root feature raw 22 thr +106.1000 | negated 22 thr -106.1000 | same 2 groups? True

contrast: a distance-based model on the same splits
5-NN raw                      test 0.912281
5-NN StandardScaler           test 0.947368
5-NN log1p (nonlinear, increasing) test 0.935673
5-NN cube (nonlinear, increasing) test 0.918129
spread of the 4 tree scores 0.0000 spread of the 4 5-NN scores 0.0351
```

A split is a question about order, not magnitude

Every question is “is $x_j \leq t$?”. Apply any strictly increasing g : then

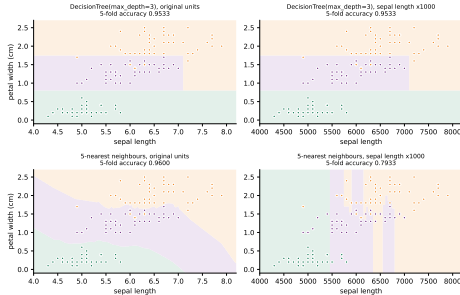
$x_j \leq t \iff g(x_j) \leq g(t)$, so **the same rows go left** with a different number printed on the node. Same partitions available, same tree.

```
a tree needs no feature scaling: any increasing transform is a no-op
tree raw leaves 16 depth 6 test 0.906433 same predictions? True
tree StandardScaler leaves 16 depth 6 test 0.906433 same predictions? True
tree log1p (nonlinear, increasing) leaves 16 depth 6 test 0.906433 same predictions? True
tree cube (nonlinear, increasing) leaves 16 depth 6 test 0.906433 same predictions? True
tree x -> -x (DECREASING) leaves 19 depth 6 test 0.900585 same predictions?
False
the decreasing case: the SAME partitions are available, but the
greedy search breaks its ties the other way round. Check the root:
root feature raw 22 thr +106.1000 | negated 22 thr -106.1000 | same 2 groups? True

contrast: a distance-based model on the same splits
5-NN raw test 0.912281
5-NN StandardScaler test 0.947368
5-NN log1p (nonlinear, increasing) test 0.935673
5-NN cube (nonlinear, increasing) test 0.918129
spread of the 4 tree scores 0.0000 spread of the 4 5-NN scores 0.0351
```

Not “similar”: **identical predictions on every test row** under an affine rescaling, a logarithm and a cube. The fifth row looks like a counterexample and is not — the root is the *same* feature at the mirrored threshold, sending the same rows each way; the trees diverge lower down only through tie-breaking.

Drawn: the tree does not move, the neighbours do



The right-hand column measures sepal length in units a thousand times smaller. Top: the tree's regions are the *same picture* — the axis numbers changed and nothing else — and the accuracy is 0.9533 in both. Bottom: 5-NN collapses from 0.9600 to 0.7933. **It does not extend to rotations:** PCA before a tree mixes columns, so the questions are no longer about single features and the invariance is gone.

Predict first #5 — an unrestricted tree on real data

ID3 and CART both stop only when a node is pure. Fit one to `load_breast_cancer` with no limits at all.

What training accuracy comes out — and what does it cost?

Predict first #5 — an unrestricted tree on real data

ID3 and CART both stop only when a node is pure. Fit one to `load_breast_cancer` with no limits at all.

What training accuracy comes out — and what does it cost?

```
breast_cancer 398 train rows, 171 test rows
unrestricted: depth 6 leaves 16 train 1.0000 test 0.9064 gap +0.0936
leaves holding a single training row: 5 of 16
```

Predict first #5 — an unrestricted tree on real data

ID3 and CART both stop only when a node is pure. Fit one to `load_breast_cancer` with no limits at all.

What training accuracy comes out — and what does it cost?

```
breast_cancer 398 train rows, 171 test rows
unrestricted: depth 6 leaves 16 train 1.0000 test 0.9064 gap +0.0936
leaves holding a single training row: 5 of 16
```

```
600 rows, 20 features, 20% of the labels deliberately flipped
unrestricted: depth 11 leaves 50 train 1.0000 test 0.6250 gap +0.3750
```

Predict first #5 — an unrestricted tree on real data

ID3 and CART both stop only when a node is pure. Fit one to `load_breast_cancer` with no limits at all.

What training accuracy comes out — and what does it cost?

```
breast_cancer 398 train rows, 171 test rows
unrestricted: depth 6  leaves 16  train 1.0000  test 0.9064  gap +0.0936
leaves holding a single training row: 5 of 16

600 rows, 20 features, 20% of the labels deliberately flipped
unrestricted: depth 11  leaves 50  train 1.0000  test 0.6250  gap +0.3750
```

Training accuracy 1.0000 from a tree is not an achievement

An unpruned tree reaches it *every time*, as long as no two rows have identical features and different labels. On `breast_cancer` the price is nine points; where a fifth of the labels are wrong by construction it is thirty-seven. **Five of the sixteen leaves hold a single patient each — those are not rules, they are copies.**

Pre-pruning: two dials, averaged over thirty splits

max_depth	leaves	train	test	gap
1	2.0	0.9275	0.8977	+0.0298
2	4.0	0.9563	0.9199	+0.0364
3	7.7	0.9750	0.9232	+0.0518
4	11.2	0.9874	0.9263	+0.0610
5	14.0	0.9946	0.9312	+0.0634
6	15.8	0.9986	0.9277	+0.0709
8	16.5	1.0000	0.9271	+0.0729
10	16.5	1.0000	0.9271	+0.0729
None	16.5	1.0000	0.9271	+0.0729
min_samples_leaf	leaves	train	test	gap
1	16.5	1.0000	0.9271	+0.0729
2	14.9	0.9904	0.9275	+0.0629
5	11.4	0.9757	0.9339	+0.0418
10	8.3	0.9607	0.9209	+0.0399
20	6.3	0.9438	0.9205	+0.0233
50	4.9	0.9275	0.8977	+0.0298

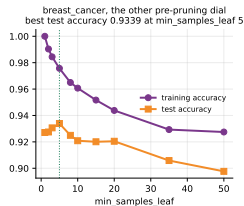
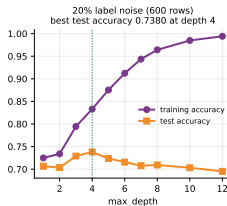
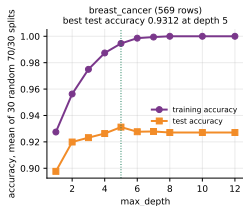
Pre-pruning: two dials, averaged over thirty splits

max_depth	leaves	train	test	gap
1	2.0	0.9275	0.8977	+0.0298
2	4.0	0.9563	0.9199	+0.0364
3	7.7	0.9750	0.9232	+0.0518
4	11.2	0.9874	0.9263	+0.0610
5	14.0	0.9946	0.9312	+0.0634
6	15.8	0.9986	0.9277	+0.0709
8	16.5	1.0000	0.9271	+0.0729
10	16.5	1.0000	0.9271	+0.0729
None	16.5	1.0000	0.9271	+0.0729

min_samples_leaf	leaves	train	test	gap
1	16.5	1.0000	0.9271	+0.0729
2	14.9	0.9904	0.9275	+0.0629
5	11.4	0.9757	0.9339	+0.0418
10	8.3	0.9607	0.9209	+0.0399
20	6.3	0.9438	0.9205	+0.0233
50	4.9	0.9275	0.8977	+0.0298

Training accuracy climbs monotonically; test accuracy **rises, peaks and falls**. `max_depth = 5` gives 0.9312; `min_samples_leaf = 5` gives 0.9339. **The train-test gap is the diagnostic, not the training score.**

Drawn: the same trade-off, and what noise does to it



max_depth 4	14.2 leaves	train 0.8330	test 0.7380	gap +0.0951
max_depth 6	32.1 leaves	train 0.9122	test 0.7157	gap +0.1965
max_depth 8	48.2 leaves	train 0.9641	test 0.7093	gap +0.2549
max_depth None	62.1 leaves	train 1.0000	test 0.6920	gap +0.3080

On breast_cancer the peak is shallow because the classes are well separated. With 20% label noise it is dramatic: 0.7380 at depth 4 against 0.6920 unrestricted, and the gap widens from 0.02 to 0.31.

Post-pruning: cost-complexity

$R_\alpha(T) = R(T) + \alpha |\tilde{T}|$ — total impurity plus a price per leaf. At $\alpha = 0$ the full tree wins; as α rises, whole subtrees stop being worth their leaf count.

```
path = DecisionTreeClassifier(random_state=0
                              ).cost_complexity_pruning_path(Xtr, ytr)
```

the path has 11 effective alphas, from 0.000000 to 0.026550

alpha	leaves	train	5-fold CV	test	
0.002492	14	0.9975	0.9171	0.9064	
0.004902	10	0.9899	0.9121	0.9123	
0.008794	6	0.9774	0.9197	0.9181	
0.008992	5	0.9724	0.9197	0.9181	
0.019591	4	0.9623	0.9247	0.9006	<-- best CV
0.025861	3	0.9322	0.9172	0.8889	
0.026550	2	0.9322	0.9172	0.8889	

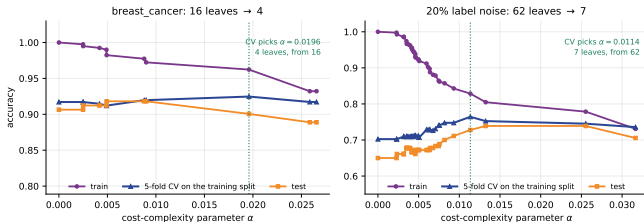
chosen by cross-validation: alpha 0.019591, 4 leaves, test 0.9006

unpruned: 16 leaves, test 0.9064

pruning removed 12 of 16 leaves and changed the test score by -0.0058

on breast_cancer the gain is in SIZE, not accuracy: report that.

Report the result you got, not the one you wanted



```
unpruned      : 50 leaves  train 1.0000  CV 0.6917  test 0.6250
best CV alpha 0.013805: 8 leaves  train 0.8500  CV 0.7444  test 0.7000
pruning removed 42 of 50 leaves and bought +0.0750 of test accuracy
```

Same code, same procedure, different data. And choose α on the **blue** curve — cross-validation inside the training split. The orange test curve is drawn so you can see whether it worked; picking the α that maximises *it* is a leak.

Summary — twelve lines

1. A decision tree is a nested set of IF–THEN rules built greedily, top-down, one question at a time; with numeric features it is a partition into axis-parallel boxes.
2. $H = -\sum p_i \log_2 p_i$ is 0 for a pure node, 1 for an even binary split, $\log_2 m$ for m equal classes. Base two makes the unit a yes/no question.
3. On the fourteen rows, $H(S) = \mathbf{0.940286}$ bits and $\text{Gini}(S) = 0.459184$.
4. The four gains: Outlook **0.246750**, Humidity 0.151836, Wind 0.048127, Temperature 0.029223. Outlook wins because it is the only attribute with a **pure child**.
5. Recursing: Sunny is split perfectly by Humidity and Rain perfectly by Wind, both at gain 0.970951. Depth 2, five leaves, Temperature never used.
6. Five leaves means five rules and nine conditions, reproducing 14 of 14 rows. On iris, two rules predicting the same class let a condition go: five rules became four, still 146 of 150.
7. Gini ranks the four attributes identically (0.116327 for Outlook) and agrees with entropy on **99.55%** of two thousand contests — but differed on 25% of breast_cancer bootstrap root splits, whose stumps still agreed on 98.4% of rows.
8. CART's best *binary* cut on Outlook scores 0.102041 against the multiway 0.116327; on the tennis table it builds depth 4 and seven leaves where ID3 builds depth 2 and five.
9. Day scores **0.940286**, the maximum possible, $3.81\times$ Outlook — and ID3 given it builds a depth-1 fourteen-leaf tree that **matches none of its own rules on a new day**.
10. Gain ratio divides by $\log_2 14 = 3.807355$, a 73.7% cut, but 0.246966 still beats 0.156428 at $n = 14$. At seventy rows gain ratio recovers the five-leaf tree while plain gain builds a seventy-leaf one.
11. A tree is invariant to any monotone transform: raw, standardised, log and cubed gave **identical predictions**, 16 leaves, test 0.906433; the same transforms moved 5-NN over 0.0351.
12. Unrestricted trees hit train **1.0000** every time, costing 0.0936 on breast_cancer and 0.3750 under 20% label noise. Cost-complexity pruning cut 16 leaves to 4 for -0.0058 — and 50 leaves to 8 for $+0.0750$. Report both.

Before the next lecture

Read

notes.pdf — the full derivation of entropy, every gain computed twice, the ID3 recursion in detail, and **practice questions with worked solutions**.

Do

Reproduce the entire fourteen-row calculation with a pen and no calculator beyond $\log_2$: $H(S)$, four gains, both recursions, the five rules. **Then do it again for the Day column** and say in one sentence why the answer is a disaster.

Next: more classifiers

Trees are one shape of decision boundary. The next lectures build others, and the vocabulary of this one — greedy, impurity, overfitting, the train-test gap — carries straight across.

A habit to start now

Before you fit anything, **look at the columns and delete the identifiers**. Row numbers, ticket numbers, patient IDs, timestamps used as labels. Information gain will happily pick every one of them.

Materials: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 4–5, 7, 14.; Zhang et al., *Dive into Deep Learning*, Ch. 5.